Could a Static Nested Class Replace a Subclass?

Access to a class compared with being a kind of that class

This addendum also started with a student's question. Right after the question about nested classes and subclasses, another student pushed the idea one step further: a static nested class can already reach the private parts of the class around it, so why bother with subclasses at all? Could a static nested class give us everything a subclass does? This is a great question, because it takes something we learned, notices what looks like a shortcut, and tests whether the shortcut holds up. That is exactly how good programmers think. The short answer is no, and the reasons are worth seeing in code. Thank you for asking it.

This addendum continues the first one, Nested Classes and Subclasses. Read that one first if you have not already. Both go with Part 1 of the inheritance notes.

A note before you begin. These are my notes from over the years, cleaned up so they are readable. They are not a study guide, and they are not gospel: not everything here appears in a lecture, and not every topic has notes yet. I write from memory and revise every semester. I also use AI to help convert documents to LaTeX, check for issues, and add explanations. I do not trust it completely, so I am still editing, and a typo or imprecise statement can survive longer than it should. Java also changes over time, though the basics we cover are resilient. If you find something that looks wrong, contradicts the book, or does not behave the way I claim when you run it, tell me. You will be doing the other students a favor, and you will understand the material better for having caught it. The textbook is the tiebreaker, and the compiler beats us both. These notes sometimes go beyond the book, but they do not replace the assigned chapters or the recorded lectures, and neither of those replaces the notes. Read the chapter, watch the lecture, and study the notes.

1. The Idea

A subclass gives `Manager` everything an `Employee` has, but it cannot touch `Employee`'s `private` fields. A static nested class *can* touch them. So what if we skipped `extends` and nested a manager class inside `Employee` instead?

Let us try it and see what we get and what we lose. Every listing in this addendum compiles as shown, unless it is marked as not compiling.

2. Trying It: A Nested Class That Is Not a Subclass

Here is a small `Employee` with a static nested class, `TeamLead`. A team lead is an employee who also earns a bonus. Because `TeamLead` does not extend `Employee`, it has to *hold* an `Employee` to have a name and a salary at all.

```
public class Employee {
    private String name;
    private double annualSalary;
```

```
    public Employee(String name, double annualSalary) {
        this.name = name;
        this.annualSalary = annualSalary;
    }

    public String getName() { return name; }

    public double getMonthlyPay() {
        return annualSalary / 12;
    }

    // Nested, but not a subclass: a TeamLead holds an Employee.
    public static class TeamLead {
        private final Employee base;
        private final double bonus;

        public TeamLead(Employee base, double bonus) {
            this.base = base;
            this.bonus = bonus;
        }

        public double getMonthlyPay() {
            return base.annualSalary / 12 + bonus / 12;   // private access works here
        }

        public String getName() {
            return base.name;                              // and here
        }
    }
}
```

```
Employee.TeamLead lead = new Employee.TeamLead(new Employee("Alan", 72000), 12000);
System.out.println(lead.getName() + ": " + lead.getMonthlyPay());
```

```
Alan: 7000.0
```

So far, it works. `TeamLead` reads `Employee`'s private fields directly, which a subclass could never do. That is the part of the idea that is true.

3. What We Lost

It is not an Employee

The whole point of the payroll program in Part 2 was one list that holds every kind of employee. A `TeamLead` cannot go in it:

```
ArrayList<Employee> staff = new ArrayList<>();
// does not compile:
staff.add(new Employee.TeamLead(new Employee("Alan", 72000), 12000));
```

```
error: incompatible types: TeamLead cannot be converted to Employee
```

The same error appears for an `Employee` variable, an `Employee[]` array, or an `Employee` parameter. Every method we wrote that works on employees, such as `totalMonthlyPay`, cannot see team leads at all.

Nothing is inherited

`TeamLead` had to write its own `getName`, which only passes the question along to `base`. The same would be true for every other method an employee has: an ID, a raise, a `toString`, an `equals`. Each one has to be written again, by hand, and kept in step with `Employee` forever.

There is no overriding

`TeamLead` has a method named `getMonthlyPay`, but it does not override anything. It is a separate, unrelated method that happens to share a name. Dynamic binding from Part 2 can never choose it, because no `Employee` reference can ever point to a `TeamLead`. Without the type, there is no polymorphism.

4. What We Actually Built: Has-a

Look at `TeamLead` again. It does not say "a team lead *is* an employee." It says "a team lead *holds* an employee." That is the has-a relationship from the earlier unit, also called composition, and Part 4 of the notes mentioned it as an alternative to inheritance.

Composition is not wrong. It is the right choice when the is-a sentence feels forced, or when one object really does wrap another. But it is a different relationship, and it gives up exactly the thing subclasses exist for: being usable anywhere the superclass is expected.

The nesting itself only added two things: the access to `private` members, and the name `Employee.TeamLead`. Neither one makes `TeamLead` a kind of `Employee`. Only `extends` does that.

5. A Class Can Be Both

Nothing stops a static nested class from also being a subclass. Here is `Supervisor`, nested inside `Employee` and extending it:

```
    // Inside Employee: nested AND a subclass.
    public static class Supervisor extends Employee {
        private final double bonus;

        public Supervisor(String name, double annualSalary, double bonus) {
            super(name, annualSalary);
            this.bonus = bonus;
        }

        @Override
        public double getMonthlyPay() {
            return super.getMonthlyPay() + bonus / 12;
        }
```

```
    }
```

```
ArrayList<Employee> staff = new ArrayList<>();
staff.add(new Employee("Ada", 60000));
staff.add(new Employee.Supervisor("Grace", 90000, 12000));   // a Supervisor is an
    Employee
for (Employee e : staff) {
    System.out.println(e.getName() + ": " + e.getMonthlyPay());
}
```

```
Ada: 5000.0
Grace: 8500.0
```

Now the list accepts it, and dynamic binding picks the override. But notice what made that possible: the `extends`, not the nesting. In fact, nesting adds very little here, and it can even trip you up. If `getMonthlyPay` in `Supervisor` tried to use the field by its simple name, as in `annualSalary / 12`, the compiler would refuse:

```
error: non-static variable annualSalary cannot be referenced from a static context
```

The simple name `annualSalary` refers to the field of an *outer* `Employee` object, and a static nested class has no outer object. That is the review notes' rule about static nested classes, showing up in a new place. Calling `super.getMonthlyPay()`, as above, is the clear way to do it.

In real programs, subclasses are almost always their own top-level classes in their own files. A nested subclass makes sense mainly for a small helper that only the outer class uses, like the `Base` and `Derived` classes inside `ConstructorOrderDemo`.

6. Summary

Table 1: A static nested class, a subclass, and a class that is both.

Question	Static nested class (`TeamLead`)	Top-level subclass (`Manager`)	Nested subclass (`Supervisor`)
Relationship	Has-a	Is-a	Is-a
Goes in an `Employee` list?	No	Yes	Yes
Inherits `Employee`'s methods?	No	Yes	Yes
Can override?	No	Yes	Yes
Reads `Employee`'s `private` fields?	Yes, through an object it holds	No	Only through an `Employee` reference, not by simple name

The one-sentence version: a static nested class gives you *access* to the outer class; a subclass

gives you its *type*, and polymorphism needs the type.

7. Try It

1. Add a `totalMonthlyPay(ArrayList<Employee> staff)` method and try to include a `TeamLead` in the total. What would you have to change, and how many places would need to change if there were five kinds of employee built like `TeamLead`?
2. Move `Supervisor` out of `Employee` into its own file, keeping `extends Employee`. Does anything else in the program have to change besides how you write its name? What does that tell you about what nesting added?
3. Describe one situation where a has-a design like `TeamLead` would be the better choice, and explain why in a sentence or two.